

Relatório de Testes Final

**Autor: João Pedro Nunes Neto
Leonardo dos Santos Silva
Lucas Gabriel Pereira Guerra
Luís Felipe Nunes da Fonseca
Luiz Phillipe de Souza Santos**

Orientador: Samuel Novais Moura Junior

João Pedro Nunes Neto
Luiz Phillipe de Souza Santos
Luís Felipe Nunes Neto
Lucas Gabriel Pereira Guerra
Leonardo dos Santos Silva

Relatório de Testes Final

Relatório técnico apresentado como requisito avaliativo da disciplina de Teste de Software – Entrega do trabalho final N2, do Curso de Engenharia de Software da Universidade Católica de Brasília.

Orientador: Prof. Samuel Novais Moura Junior

Brasília – DF

2026

Sumário

1.	Introdução	4
2.	Planejamento de Testes.....	4
2.1	Objetivo.....	4
2.2	Escopo	4
2.3	Ferramentas e Estratégia	4
2.4	Requisito de Ambiente	5
2.5	Resumo da Suíte de Testes	5
3.	Descrição dos Casos de Teste	6
4.	Resultados das Execuções e Evidências	11
4.1	Resumo dos Resultados.....	11
4.2	Evidências — Testes Unitários e de Controller	11
4.3	Evidências — Testes de API (Postman)	12
4.4	Evidências — Testes E2E (Playwright)	17
5.	Conclusão e Lições Aprendidas	21
5.1	Conclusão	21
5.2	Lições Aprendidas	22
6.	Repositório no GitHub	23

RollCore — RPG Skill Checker & Companion

Equipe 8 — Teste de Software | Mai/2026

1. Introdução

Neste trabalho foi desenvolvida uma plataforma digital voltada para jogadores de RPG de mesa, com o intuito de facilitar a organização de fichas de personagem e a realização de rolagens de dados virtuais. Cada usuário autenticado pode criar e gerenciar fichas de múltiplos personagens e rolar dados virtuais para uso nas sessões de jogo.

O backend foi desenvolvido em Java com Spring Boot, utilizando JUnit 5 e Mockito para testes unitários e de controller, Postman para testes de API em ambiente real e Playwright para testes End-to-End. O frontend web foi construído com React e TypeScript.

O repositório completo do projeto está disponível em: <https://github.com/Takesh0s/RollCore>

2. Planejamento de Testes

2.1 Objetivo

Garantir que as funcionalidades do RollCore funcionem corretamente e de forma integrada, identificando defeitos antes da entrega final.

2.2 Escopo

- Autenticação: registro, login e refresh de token JWT
- CRUD de fichas de personagem (criar, listar, buscar por ID, editar e excluir)
- Rolagem de dados virtuais (D4, D6, D8, D10, D12, D20, D100) e fórmulas personalizadas
- Lógica de regras D&D 5e: modificadores de atributo, bônus de proficiência, slots de magia, save DC e bônus de ataque
- Validações de formulário e mensagens de erro nos endpoints REST

2.3 Ferramentas e Estratégia

Nível	Ferramenta	Estratégia	Meta
Unitário	JUnit 5 + Mockito	Caixa branca — classes isoladas, sem Spring Context	Cobertura $\geq$ 80%

API / Controller	Spring WebMvcTest + MockMvc	Caixa preta — camada web com filtros bypassados	Status codes corretos
API (Live)	Postman / Newman	Caixa preta — chamadas HTTP em ambiente real	Contratos validados
E2E	Playwright (Chromium)	Caixa preta — fluxos completos no navegador	6 cenários cobertos

2.4 Requisito de Ambiente

ATENCAO - JDK 21 obrigatorio: Este projeto e compilado e executado EXCLUSIVAMENTE com Java 21 (JDK 21 LTS). Versoes superiores (22, 23, 24+) NAO sao suportadas e causarao falhas de compilacao ou erros de runtime. Confirme que JAVA_HOME aponta para o JDK 21 antes de executar qualquer teste.

2.5 Resumo da Suíte de Testes

A suíte compreende 120 casos de teste distribuídos em 6 arquivos Java (JUnit 5) e 1 coleção Postman, cobrindo 3 histórias de usuário (HU-01, HU-02 e HU-03), com tempo total de execução do backend de aproximadamente 731 ms.

Arquivo de Teste	Tipo	Testes	HU Relacionada
AuthControllerTest	API (Controller / WebMvcTest)	8	HU-01
CharacterControllerTest	API (Controller / WebMvcTest)	7	HU-02
DndEngineTest	Unitário (sem Spring Context)	45	HU-02
CharacterServiceTest	Unitário (Mockito, sem Spring Context)	22	HU-02
DiceServiceTest	Unitário (Mockito, sem Spring Context)	17	HU-03
SpellServiceTest	Unitário (Mockito, sem Spring Context)	9	HU-02
RollCoreE2E	E2E (Playwright)	6	HU-01, HU-02, HU-03
RollCore.postman_Collection.json	API (Ambiente real — Postman)	6	HU-01, HU-03
TOTAL	—	120	HU-01, HU-02, HU-03

Nota: DndEngineTest e CharacterServiceTest utilizam @ParameterizedTest com @CsvSource e @ValueSource. Cada linha/valor dos parâmetros gera um caso de teste independente no JUnit, por isso o total de casos de teste difere do número de métodos de teste no código-fonte.

3. Descrição dos Casos de Teste

A tabela abaixo apresenta uma visão consolidada dos 120 casos de teste implementados, organizados por arquivo e história de usuário. Para a descrição detalhada de cada caso, incluindo pré-condições, passos de execução e critérios de aceite, consulte o Plano de Testes v1.2 (documento separado).

ID	Classe	Método	Tipo	H U	Descrição	Entrada	Saída Esperada
HU-01 — Autenticação (AuthControllerTest) — 8 testes							
CT-AUT H-01	AuthControllerTest	registerSuccess	API Controller	H U-01	Registro com payload válido retorna 201 e accessToken	username, email, senha válidos	HTTP 201 + accessToken
CT-AUT H-02	AuthControllerTest	registerEmailConflict	API Controller	H U-01	E-mail duplicado retorna 409	E-mail já cadastrado	HTTP 409
CT-AUT H-03	AuthControllerTest	registerBadUsername	API Controller	H U-01	Username inválido retorna 400	'Bad Name!'	HTTP 400
CT-AUT H-04	AuthControllerTest	registerWeakPassword	API Controller	H U-01	Senha fraca retorna 400	senha: 'weak'	HTTP 400
CT-AUT H-05	AuthControllerTest	loginSuccess	API Controller	H U-01	Login com credenciais válidas retorna 200 e token	email+senha corretos	HTTP 200 + accessToken
CT-AUT H-06	AuthControllerTest	loginBadCredentials	API Controller	H U-01	Senha errada retorna 401	senha incorreta	HTTP 401
CT-AUT H-07	AuthControllerTest	refreshSuccess	API Controller	H U-01	Refresh token válido retorna 200 e novo token	refresh token válido	HTTP 200 + novo token
CT-AUT H-08	AuthControllerTest	refreshExpired	API Controller	H U-01	Refresh token expirado retorna 401	token expirado	HTTP 401
HU-01 — Autenticação E2E (RollCoreE2E) — 4 testes							
E2E-AUT H-01	RollCoreE2E	—	E2E (Web)	H U-01	Login completamente inválido não autoriza acesso	Email e senha inválidos	Acesso negado

E2E-AUT H-02	RollCoreE2E	—	E2E (Web)	H U-01	Login com senha inválida não autoriza acesso	Email válido, senha errada	Acesso negado
E2E-AUT H-03	RollCoreE2E	—	E2E (Web)	H U-01	Criação de conta permite acesso automático	Email + senha novos	Acesso ao sistema
E2E-AUT H-04	RollCoreE2E	—	E2E (Web)	H U-01	Login válido concede acesso ao sistema	Email + senha corretos	Acesso ao sistema
HU-01 — Autenticação (Postman Live) — 3 testes							
PSM N-AUT H-01	Postman	Create First User	API (Live)	H U-01	Registro válido retorna 201 e token	username, email, senha	HTTP 201 + token
PSM N-AUT H-02	Postman	EmailConflict	API (Live)	H U-01	E-mail duplicado retorna 409	Email já cadastrado	HTTP 409
PSM N-AUT H-03	Postman	Login	API (Live)	H U-01	Login válido retorna 200 e token	Email + senha corretos	HTTP 200 + token
HU-02 — Personagem Controller (CharacterControllerTest) — 7 testes							
CT-CHAR-01	CharacterControllerTest	listCharacters	API Controller	H U-02	GET /characters retorna 200 e lista do usuário	JWT válido	HTTP 200 + array
CT-CHAR-02	CharacterControllerTest	createSuccess	API Controller	H U-02	POST /characters com payload válido retorna 201	Payload completo	HTTP 201; name='Aragorn'
CT-CHAR-03	CharacterControllerTest	createMissingName	API Controller	H U-02	Name em branco retorna 400	name: "	HTTP 400
CT-CHAR-04	CharacterControllerTest	createLevelTooHigh	API Controller	H U-02	Level=21 retorna 400	level: 21	HTTP 400
CT-CHAR-05	CharacterControllerTest	getByIdFound	API Controller	H U-02	GET por ID retorna 200 quando existe	ID existente	HTTP 200
CT-CHAR-06	CharacterControllerTest	getByIdNotFound	API Controller	H U-02	GET por ID retorna 404 quando não existe	UUID inexistente	HTTP 404
CT-CHAR-07	CharacterControllerTest	deleteCharacter	API Controller	H U-02	DELETE retorna 204 e invoca service.delete()	ID existente	HTTP 204

HU-02 — Engine D&D 5e (DndEngineTest) — 45 testes parametrizados							
CT-ENG-01~09	DndEngineTest	modifierTable	Unitário	HU-02	calcMod(score) = floor((score-10)/2) — 9 variações (scores 1 a 30)	scores: 1,8,9,10,11,12,15,20,30	mods: -5,-1,-1,0,0,1,2,5,10
CT-ENG-10~19	DndEngineTest	profBonusTable	Unitário	HU-02	profBonus(level) conforme tabela PHB — 10 variações (níveis 1 a 20)	levels: 1,4,5,8,9,12,13,16,17,20	bonus: 2,2,3,3,4,4,5,5,6,6
CT-ENG-20~30	DndEngineTest	spellSlots*	Unitário	HU-02	Slots de magia para lançadores completos, meio-lançadores e não-lançadores	Diversas classes e níveis	Slots conforme PHB ou null
CT-ENG-31~33	DndEngineTest	resolveCaster*	Unitário	HU-02	Resolve CasterType e atributo de magia para subclasses arcanas	Cavaleiro Arcano, Trapaceiro Arcano	CasterType.THIRD, 'INT'
CT-ENG-34~37	DndEngineTest	warlockSlots*	Unitário	HU-02	Pact Magic do Warlock — níveis 1, 5, 20 e classe não-warlock	Warlock lvl 1/5/20; Mago	Slots corretos ou null
CT-ENG-38~43	DndEngineTest	spellSaveDC/Attack/Validation	Unitário	HU-02	SpellSaveDC, spellAttackBonus, isValidClass, isValidRace	Diversas classes e atributos	DCs, bonus e validações corretos
CT-ENG-44	DndEngineTest	clerigoFullCaster	Unitário	HU-02	resolveCasterType("Clérigo", null) = FULL; resolveSpellAbility = "WIS"	classe: "Clérigo", subclasse: null	CasterType.FULL, "WIS"
CT-ENG-45	DndEngineTest	druidaLevel3	Unitário	HU-02	Slots de nível 3 do Druida = 4/2/0 conforme PHB	Druida nível 3	slots[0]=4, slots[1]=2, slots[2]=0
HU-02 — Serviço de Personagem (CharacterServiceTest) — 22 testes							
CT-SVC-01~06	CharacterServiceTest	acModifierTable	Unitário	HU-02	AC = 10 + mod(DEX) — 6 variações (DEX 1 a 20)	DEX: 1,8,10,12,16,20	AC: 5,9,10,11,13,15
CT-SVC-07	CharacterServiceTest	acIsComputedServerSide	Unitário	HU-02	AC calculado server-side; não aceita valor do cliente	DEX=14, Guerreiro, level:1	AC: 12

CT-SVC-08~10	CharacterServiceTest	spellSlots*	Unitário	HU-02	Slots populados pelo engine; null para não-lançadores; Pact Magic do Warlock	Mago, Guerreiro, Warlock	Slots corretos ou null
CT-SVC-11~12	CharacterServiceTest	saveIsCalledOnce / tempHp	Unitário	HU-02	save() chamado 1x; tempHp null tratado como 0	Bardo; tempHp null	save() invocado; tempHp=0
CT-SVC-13~17	CharacterServiceTest	list/get/forbidden/not Found	Unitário	HU-02	listByUser, getById, ForbiddenException, NotFoundException	Diversos usuários e IDs	Listas corretas; exceções adequadas
CT-SVC-18~21	CharacterServiceTest	update/delete*	Unitário	HU-02	update recalcula AC e slots; delete verifica posse	DEX 10→16; Mago lvl 1→5	AC/slots recalculados; exceção se outro usuário
CT-SVC-22	CharacterServiceTest	throwsForbiddenOnUpdateOther	Unitário	HU-02	update() lança ForbiddenException para personagem de outro usuário; save() nunca invocado	Personagem de outro usuário	ForbiddenException; save() não chamado

HU-02 — Personagem E2E e Postman — 2 testes

E2E-CHAR-01	RollCoreE2E	—	E2E (Web)	HU-02	Criar personagem e editar HP para 38	Personagem novo + edição HP	Personagem com HP=38
PSM-N-CHAR-01	Postman	Create a char	API (Live)	HU-02	Criar personagem com todos os atributos	Payload completo	Personagem criado com sucesso

HU-02 — Magias (SpellServiceTest) — 9 testes

CT-SPELL-01	SpellServiceTest	filterReturnsSpells	Unitário	HU-02	filter() retorna SpellResponse mapeada	classe:'Mago', nível:3	Lista com 'Bola de Fogo'
CT-SPELL-02~03	SpellServiceTest	filterNull/BlankClass	Unitário	HU-02	Parâmetros null/branco retornam lista vazia ou repassam null	null; ' '	Lista vazia; repo chamado com null
CT-SPELL-	SpellServiceTest	add* (sucesso/duplicado/proibido)	Unitário	HU-02	addToCharacter: sucesso, ConflictException,	IDs válidos / duplicados / outro usuário	Salvo; ConflictEx; ForbiddenEx

04~06					ForbiddenException		
CT-SPELL-07~09	SpellServiceTest	remove* / listForCharacter	Unitário	HU-02	removeFromCharacter: sucesso, NotFoundException; listForCharacter	IDs válidos / não associados	Deletado; NotFoundException; Lista mapeada
HU-03 — Dados (DiceServiceTest) — 17 testes							
CT-DICE-01	DiceServiceTest	rollFormula2d6plus3	Unitário	HU-03	Rola 2d6+3 e verifica estrutura completa	formula: '2d6+3'	rolls.size=2, mod=3, total correto
CT-DICE-02~05	DiceServiceTest	validFormulas	Unitário	HU-03	Fórmulas válidas aceitas: 1d20, 4d4-1, 1d100, 99d4+99	Fórmulas válidas	id not null
CT-DICE-06	DiceServiceTest	d20BoundsCheck	Unitário	HU-03	1d20 sempre retorna [1,20] em 100 iterações (SecureRandom)	1d20 × 100	total ∈ [1, 20]
CT-DICE-07~13	DiceServiceTest	invalidFormulas	Unitário	HU-03	Fórmulas inválidas lançam InvalidFormulaException: 2d7, 0d6, abc, d20, 2d, 2d6+, 100d6	Fórmulas inválidas	InvalidFormulaException; repo não chamado
CT-DICE-14~15	DiceServiceTest	getHistory*	Unitário	HU-03	getHistory() retorna últimas 50 rolagens ou lista vazia	userId com / sem rolagens	Lista mapeada ou vazia
CT-DICE-16	DiceServiceTest	rollFormulaNegativeMod	Unitário	HU-03	Fórmula 1d8-2 é aceita; mod=-2 e total = dado - 2	formula: '1d8-2'	mod=-2, total correto
CT-DICE-17	DiceServiceTest	d4BoundsCheck	Unitário	HU-03	1d4 sempre retorna [1,4] em 100 iterações (SecureRandom)	1d4 × 100	total ∈ [1, 4]
HU-03 — Dados E2E e Postman — 3 testes							
E2E-DICE-01	RollCoreE2E	—	E2E (Web)	HU-03	Rolar 1d20 via fórmula retorna sucesso	formula: '1d20'	Dado rolado com sucesso
PSM-N-DICE-01	Postman	5 d20 Rolls	API (Live)	HU-03	Rolar 5d20 retorna 5 dados	formula: '5d20'	5 dados com sucesso

PSM N- DIC E-02	Postman	History	API (Live)	H U- 03	GET /dice/history retorna histórico não vazio	Histórico com rolagens	Lista não vazia
--------------------------	---------	---------	---------------	---------------	---	---------------------------	-----------------

* Grupos de testes parametrizados. Cada variação constitui um caso independente no JUnit 5. O detalhamento linha a linha encontra-se no Plano de Testes v1.2.

4. Resultados das Execuções e Evidências

4.1 Resumo dos Resultados

Suite	Total	Passou	Falhou	Observação
Testes Unitários (JUnit 5)	93	93	0	DndEngineTest, CharacterServiceTest, DiceServiceTest, SpellServiceTest
Testes de API Controller (WebMvcTest)	15	15	0	AuthControllerTest, CharacterControllerTest
Testes de API Live (Postman)	6	6	0	RollCore.postman_Collection.json
Testes E2E (Playwright)	6	6	0	RollCoreE2E — Chromium
TOTAL	120	120	0	Execução total backend ≈ 731 ms

4.2 Evidências — Testes Unitários e de Controller

[PRINT / EVIDÊNCIA] — Relatório JaCoCo: cobertura de código do backend

```

✓ rollcore-backend [test]: At 31/05/26, 17:34 8 sec, 328 ms
[INFO] Running com.rollcore.service.DiceServiceTest$RollInvalidTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in com.rollcore.service.DiceS
[INFO] Running com.rollcore.service.DiceServiceTest$RollValidTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.023 s -- in com.rollcore.service.DiceS
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 s -- in com.rollcore.service.DiceS
[INFO] Running com.rollcore.engine.DndEngineTest
[INFO] Running com.rollcore.engine.DndEngineTest$SpellSaveDCTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.083 s -- in com.rollcore.engine.DndEng
[INFO] Running com.rollcore.engine.DndEngineTest$WarlockSlotsTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.082 s -- in com.rollcore.engine.DndEng
[INFO] Running com.rollcore.engine.DndEngineTest$ThirdCasterSlotsTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 s -- in com.rollcore.engine.DndEng
[INFO] Running com.rollcore.engine.DndEngineTest$SpellSlotsTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in com.rollcore.engine.DndEng
[INFO] Running com.rollcore.engine.DndEngineTest$ProfBonusTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s -- in com.rollcore.engine.DndEng
[INFO] Running com.rollcore.engine.DndEngineTest$CalcModTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in com.rollcore.engine.DndEng
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s -- in com.rollcore.engine.DndEng
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 103, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.267 s
[INFO] Finished at: 2026-05-31T17:34:18-03:00
[INFO] -----

```

[illegible]

PRINT / EVIDÊNCIA] — PSMN-AUTH-01: POST /auth/register → HTTP 201 + accessToken

The screenshot displays the Postman interface for a REST client. A POST request is configured to the endpoint `http://localhost:8080/auth/register`. The request body is raw JSON, containing:

```
{
  "username": "PlayerOne",
  "email": "player@rollcore.com",
  "password": "Senha123"
}
```

The response status is **PASSED** with a status code of 201. The response headers include:

- `Vary: "origin,access-control-request-method,access-control-request-headers,accept-encoding"`
- `X-Content-Type-Options: "nosniff"`
- `X-XSS-Protection: "0"`
- `Cache-Control: "no-cache, no-store, max-age=8, must-revalidate"`
- `Pragma: "no-cache"`
- `Expires: "0"`
- `X-Frame-Options: "DENY"`
- `Content-Encoding: "gzip"`
- `Content-Type: "application/json"`
- `Transfer-Encoding: "chunked"`
- `Date: "Sun, 31 May 2026 20:37:04 GMT"`
- `Keep-Alive: "timeout=60"`
- `Connection: "keep-alive"`

The response body is a JSON object containing an access token and other information:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ9Lj26KRYXN3a70g2gxDrLnAyGK_vwv6-4s-MC5mMMtcB0EyTlIEB-ph2X-p0kViMfdg",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ9Lj26KRYXN3a70g2gxDrLnAyGK_vwv6-4s-MC5mMMtcB0EyTlIEB-ph2X-p0kViMfdg",
  "userId": "cb57bb57-f662-4b20-bb2b-d6b6147e5409",
  "email": "player@rollcore.com",
  "username": "PlayerOne"
}
```

[PRINT / EVIDÊNCIA] — PSMN-AUTH-02: POST /auth/register (e-mail duplicado) → HTTP 409

The screenshot displays the Postman interface for a POST request to `http://localhost:8080/auth/register`. The 'Test Results' tab shows a 'PASSED' status with the message 'Status code is 409'. The 'Scripts' tab contains the following code:

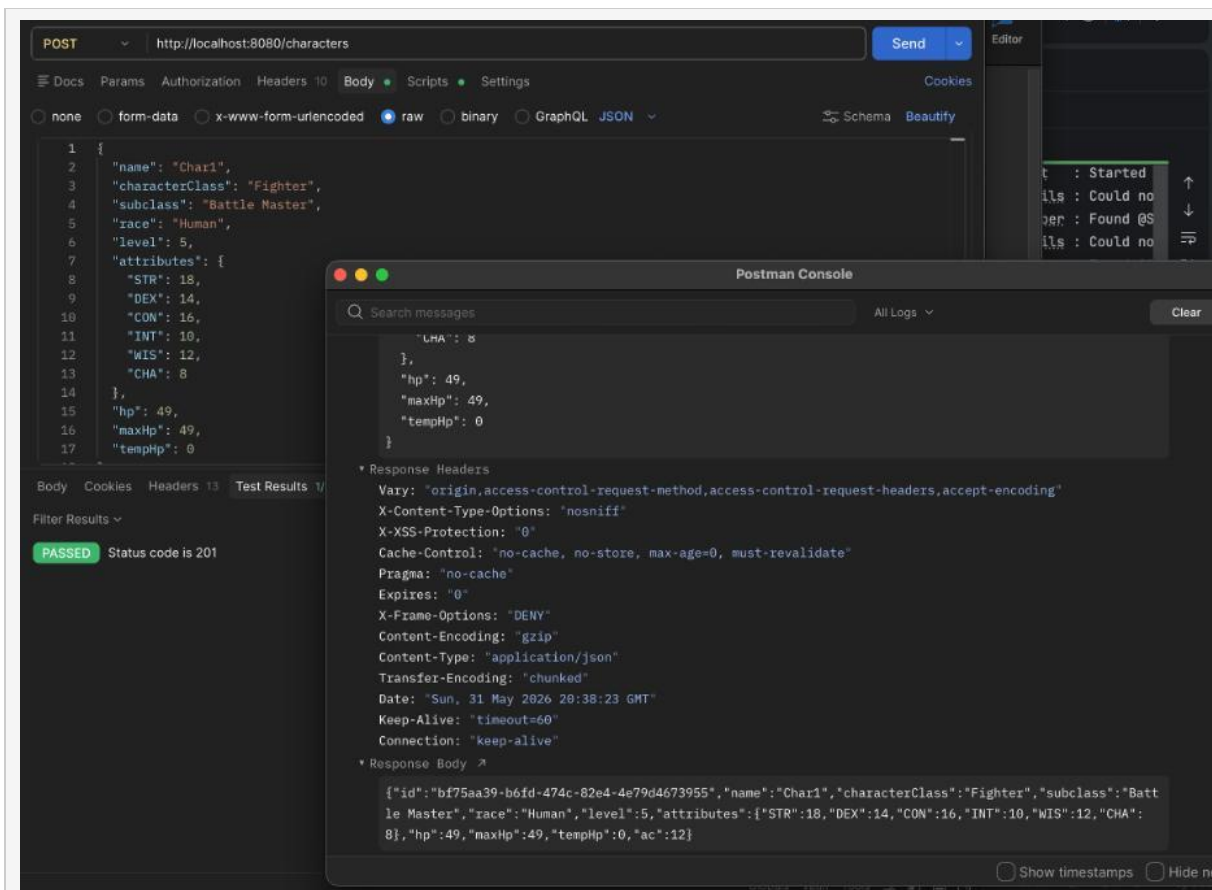
```
1 var jsonData = pm.response.json();
2 pm.test("Status code is 409", function () {
3   pm.response.to.have.status(409);
4 });
```

The 'Postman Console' is open, showing the response details:

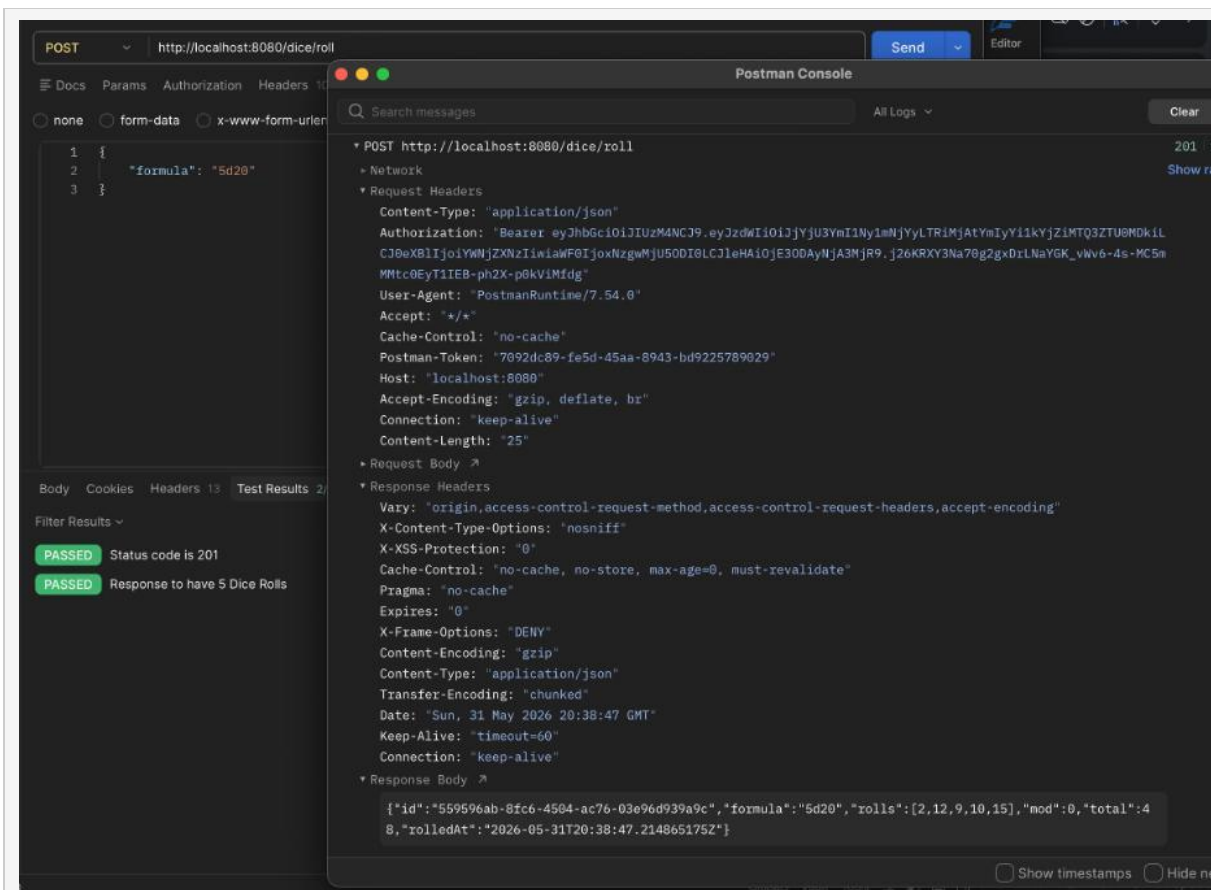
- Host: "localhost:8080"
- Accept-Encoding: "gzip, deflate, br"
- Connection: "keep-alive"
- Content-Length: "95"
- Response Headers:
 - Vary: "Origin"
 - Vary: "Access-Control-Request-Method"
 - Vary: "Access-Control-Request-Headers"
 - X-Content-Type-Options: "nosniff"
 - X-XSS-Protection: "0"
 - Cache-Control: "no-cache, no-store, max-age=0, must-revalidate"
 - Pragma: "no-cache"
 - Expires: "0"
 - X-Frame-Options: "DENY"
 - Content-Type: "application/problem+json"
 - Transfer-Encoding: "chunked"
 - Date: "Sun, 31 May 2026 20:37:29 GMT"
 - Keep-Alive: "timeout=60"
 - Connection: "keep-alive"
- Response Body:

```
{
  "type": "urn:rollcore:conflict",
  "title": "Conflict",
  "status": 409,
  "detail": "Email is already registered. Please use a different email or log in.",
  "instance": "/auth/register"
}
```

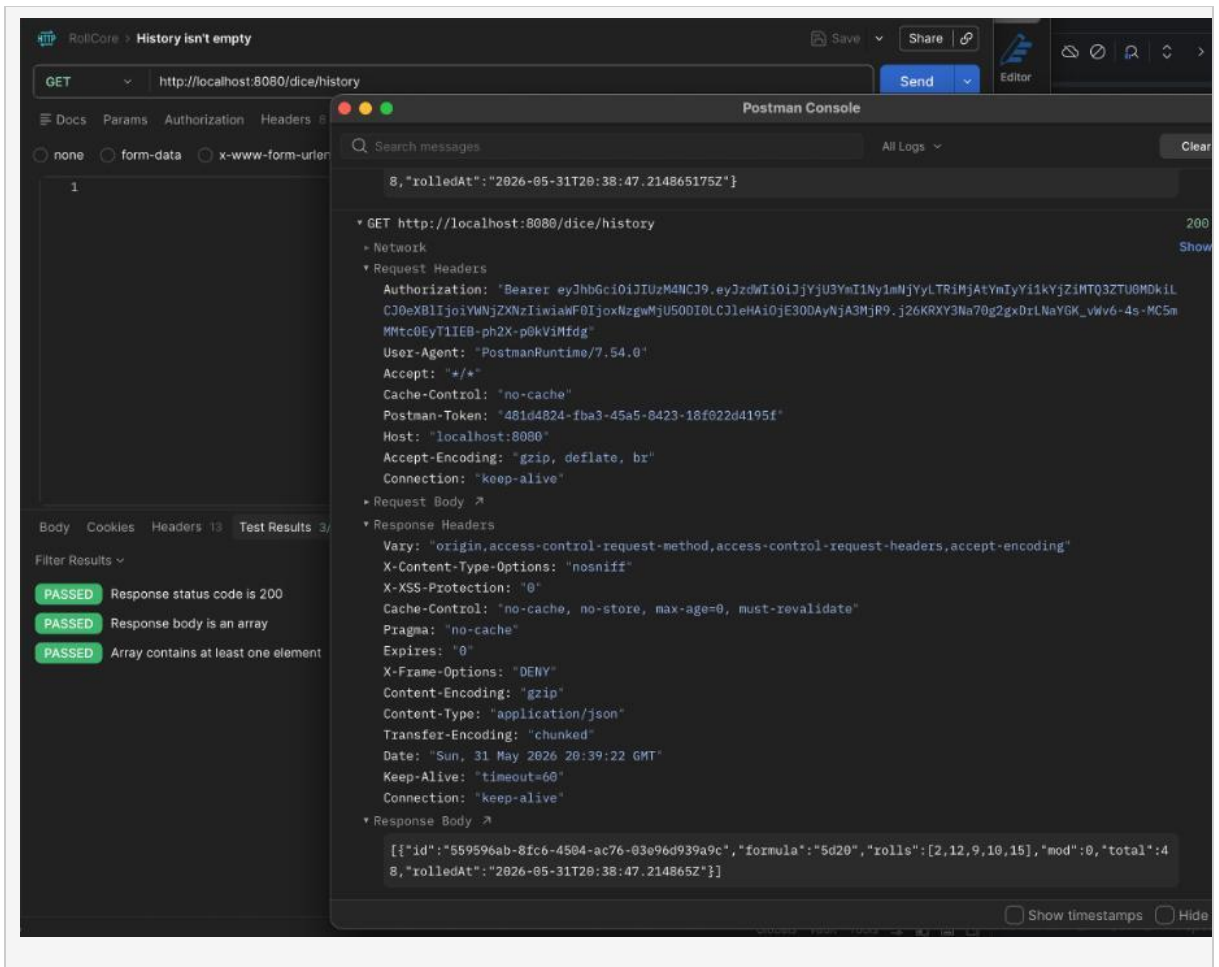
[PRINT / EVIDÊNCIA] — PSMN-AUTH-03: POST /auth/login → HTTP 200 + accessToken



[PRINT / EVIDÊNCIA] — PSMN-DICE-01: fórmula 5d20 → 5 dados rolados



[PRINT / EVIDÊNCIA] — PSMN-DICE-02: GET /dice/history → histórico não vazio



4.4 Evidências — Testes E2E (Playwright)

[PRINT / EVIDÊNCIA] — Playwright UI: visão geral dos 6 testes executados

Search tests	All 16	✓ Passed 16	✗ Failed 0	⚠ Flaky 0	⏸ Skipped 0	⚙
31/05/2025, 17:39:57 Total time: 45.6s						
▼ RollCore.spec.ts						
✗	test Criacao de personagem, e verificacao da Engine	firefox	30.2s	View Trace		
✗	Rolagem de dados	firefox	30.2s	View Trace		
✓	Login completamente Invalido	chromium	7.3s	View Trace		
✓	Login Parcialmente Invalido	chromium	7.1s	View Trace		
✓	Login Valido	chromium	8.1s	View Trace		
✓	Criacao de conta e autoLogin	chromium	7.8s	View Trace		
✓	test Criacao de personagem, e verificacao da Engine	chromium	10.8s	View Trace		
✓	Rolagem de dados	chromium	3.4s	View Trace		
✓	Login completamente Invalido	firefox	2.5s	View Trace		
✓	Login Parcialmente Invalido	firefox	2.5s	View Trace		
✓	Login Valido	firefox	3.9s	View Trace		
✓	Criacao de conta e autoLogin	firefox	4.5s	View Trace		
✓	Login completamente Invalido	webkit	2.2s	View Trace		
✓	Login Parcialmente Invalido	webkit	2.1s	View Trace		
✓	Login Valido	webkit	3.3s	View Trace		
✓	Criacao de conta e autoLogin	webkit	3.1s	View Trace		
✓	test Criacao de personagem, e verificacao da Engine	webkit	6.2s	View Trace		
✓	Rolagem de dados	webkit	3.4s	View Trace		

[PRINT / EVIDÊNCIA] — E2E-AUTH-01/02: tentativas de login inválido negadas

Playwright RollCore.spec.ts:51 - Login completamente Invalido

Actions Metadata 5 hidden

Filter actions:

- Before Hooks 1.6s
- Navigate to "/" 681ms
- Click 293ms
- Fill "emailerrado@email.co..." 34ms
- Click 391ms
- Fill "senhaerrada123" 42ms
- Click 277ms
- Check 153ms
- Click 277ms
- After Hooks 1.2s
- Fixture "page" 1ms
- Fixture "context" 1.2s

Locator Call Log Errors Console Network Source Attachments

Filter networks: All Fetch HTML JS CSS Font Image

Name: euuwww.amvu.gr/em/rw/

Headers: Payload Response

Method: POST

Status Code: 401

Start: 5.3s

Duration: 50ms

http://localhost:5173/

ROLLCORE

E-MAIL: emailerrado@email.co

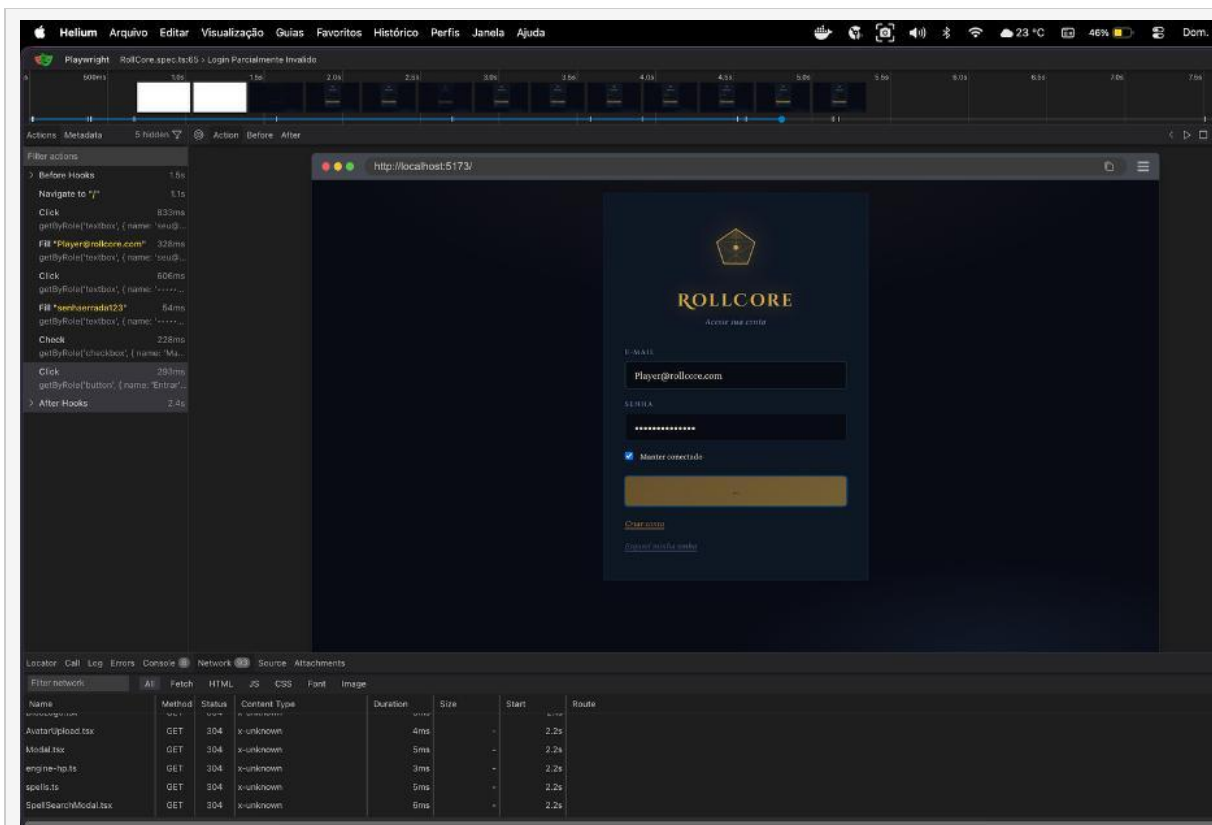
SENHA: *****

Entrar

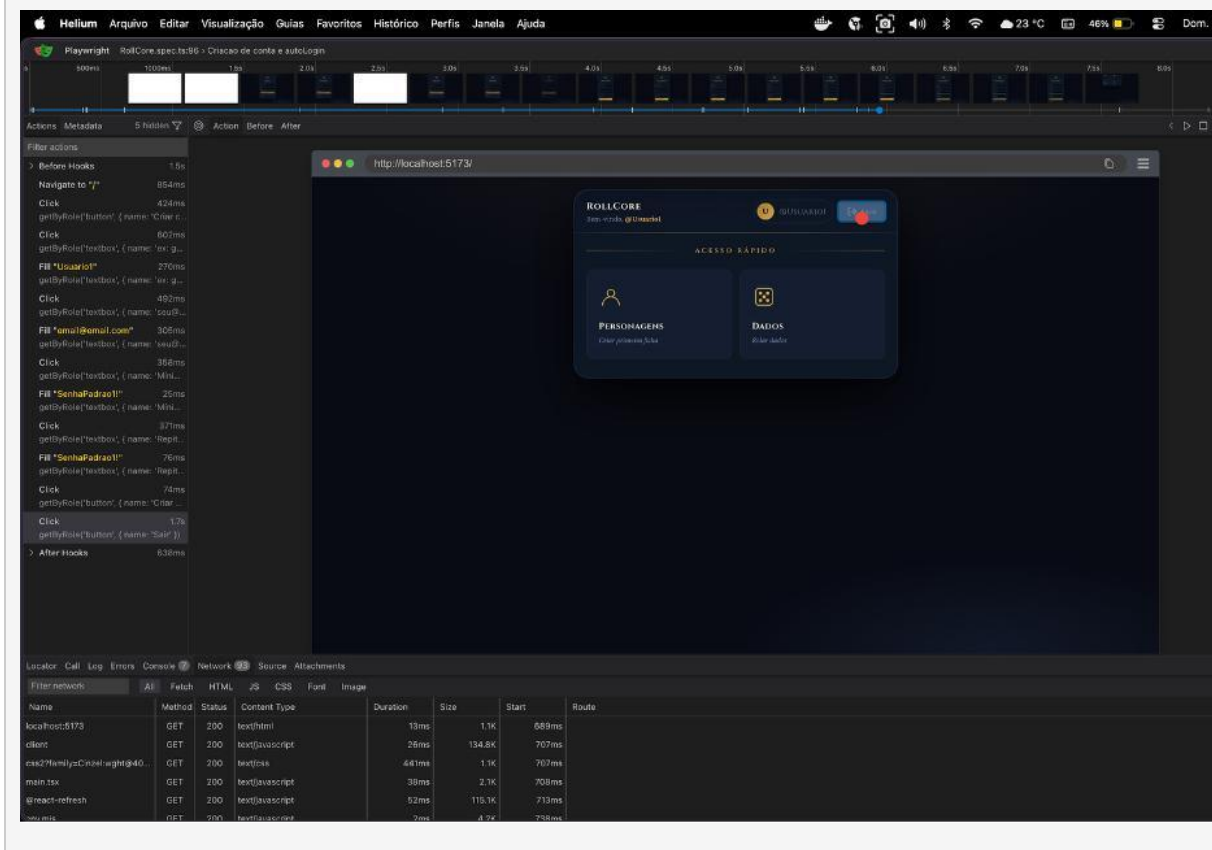
Master conectado

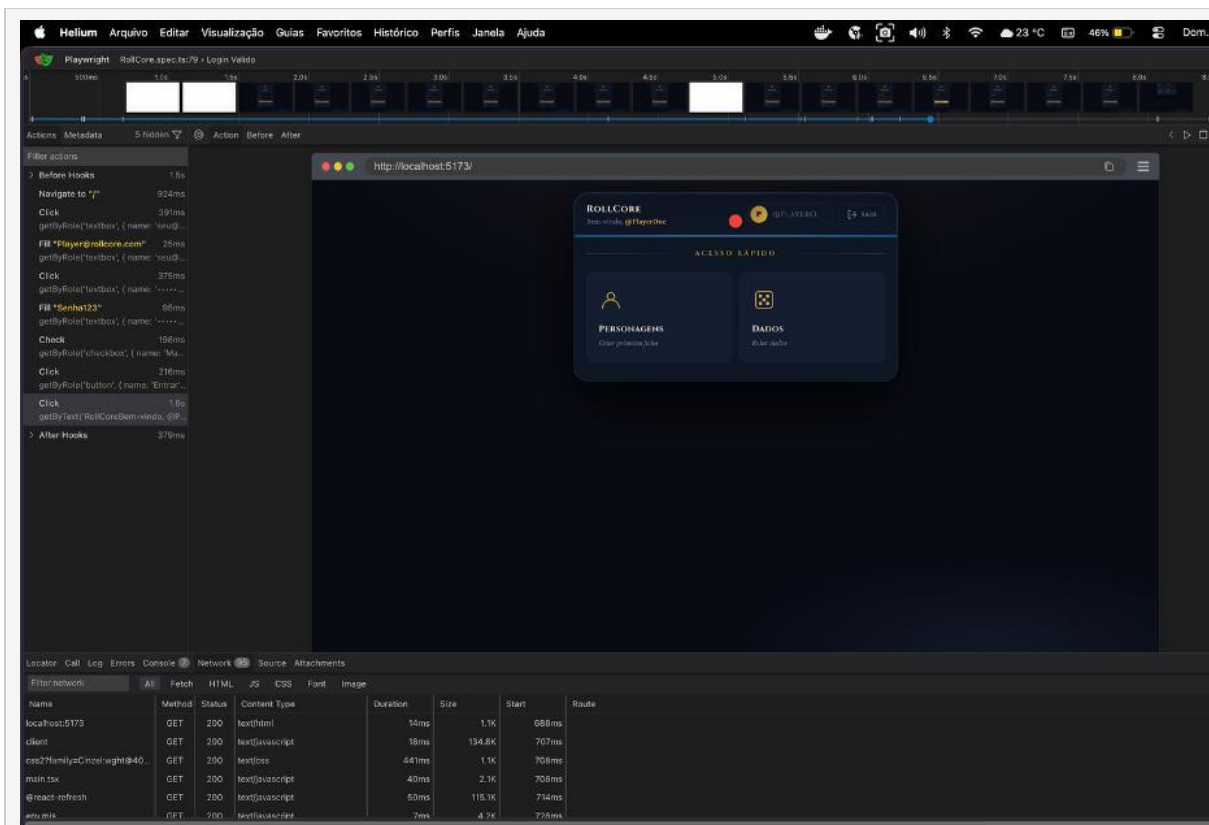
Entrar

localhost:5173

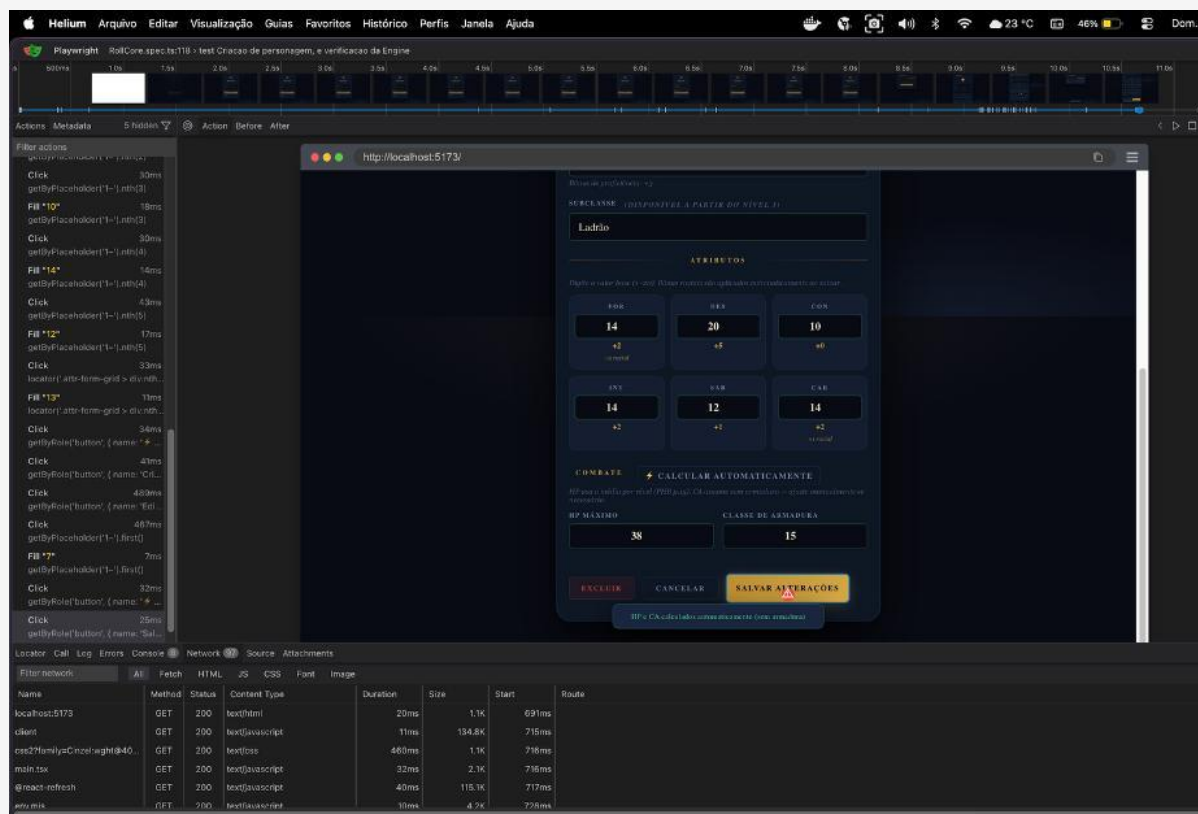


[PRINT / EVIDÊNCIA] — E2E-AUTH-03/04: cadastro e login válido — acesso concedido

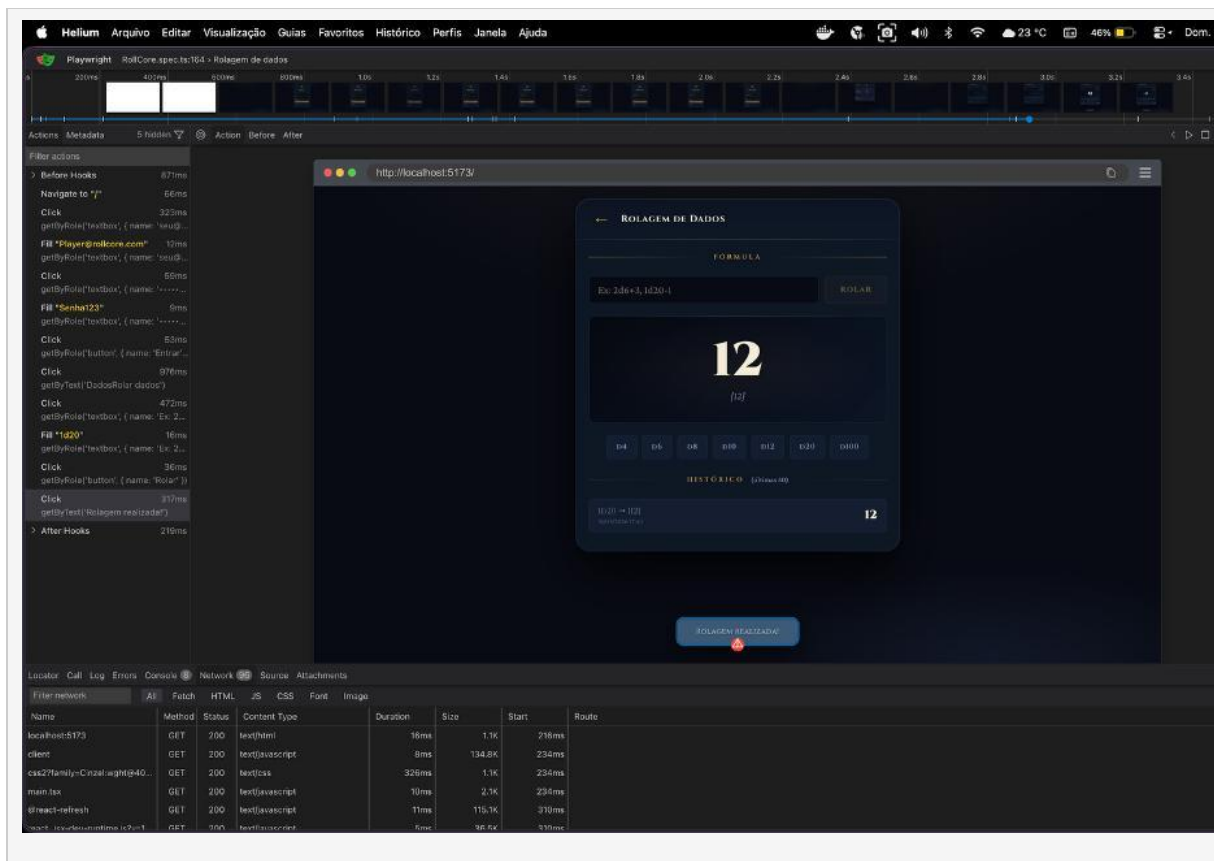




[PRINT / EVIDÊNCIA] — E2E-CHAR-01: criação e edição de personagem com HP=38



[PRINT / EVIDÊNCIA] — E2E-DICE-01: rolagem de 1d20 com sucesso



5. Conclusão e Lições Aprendidas

5.1 Conclusão

O projeto RollCore foi desenvolvido com o objetivo de oferecer uma plataforma digital completa para jogadores de RPG de mesa, permitindo a criação e o gerenciamento de fichas de personagem e a realização de rolagens de dados virtuais. Ao longo do ciclo de desenvolvimento, a equipe adotou uma abordagem estruturada de testes de software em três níveis distintos: unitários, de API e E2E.

Os testes unitários desenvolvidos com JUnit 5 e Mockito garantiram a correteza isolada das classes e serviços do backend, validando desde a lógica de autenticação até o motor completo de regras D&D 5e (modificadores, bônus de proficiência, slots de magia, Pact Magic, spell save DC e spell attack bonus). A cobertura de código foi monitorada com JaCoCo, com meta estabelecida em 80%.

Os testes de API, tanto com WebMvcTest (camada de controller isolada) quanto com Postman (ambiente real) verificaram o comportamento dos endpoints REST, confirmando que os contratos de comunicação entre frontend e backend estão

corretos e consistentes em todos os fluxos de autenticação, gestão de personagens e rolagem de dados.

Os testes E2E implementados com Playwright simularam a interação real do usuário com o sistema nos fluxos críticos: autenticação, criação/edição de personagens e rolagem de dados. Os 120 casos de teste executados cobriram integralmente as três histórias de usuário (HU-01, HU-02 e HU-03), com 100% de aprovação e tempo total de execução do backend de aproximadamente 731 ms.

A experiência demonstrou que a adoção de uma estratégia de testes em múltiplos níveis aumenta significativamente a confiança na qualidade do software, reduz o risco de regressões e facilita a manutenção ao longo do tempo.

5.2 Lições Aprendidas

- Planejar os casos de teste antes de implementá-los é fundamental: a definição prévia de IDs, passos, entradas e critérios de aceitação evita retrabalho e torna a execução mais objetiva e rastreável.
- O padrão Page Object Model (POM) adotado no Playwright foi determinante para a organização dos testes E2E: separar a lógica de navegação da lógica de verificação facilita a manutenção e permite reutilização quando os componentes da interface mudam.
- Testes de API com Postman se mostraram muito eficientes para identificar inconsistências nos retornos dos endpoints como campos ausentes no response body e status codes incorretos antes mesmo da integração com o frontend.
- A sincronização entre seletores do Playwright e os componentes React foi um desafio recorrente. Adicionar atributos data-testid nos componentes desde o início do desenvolvimento reduz drasticamente o esforço de escrita e manutenção dos testes E2E.
- A restrição ao JDK 21 evidenciou a importância de documentar e fixar o ambiente de execução no início do projeto: versões superiores do JDK introduzem mudanças de compatibilidade que podem causar falhas silenciosas. O pipeline de CI/CD foi configurado para JDK 21 desde o início.

- A automação de testes integrada ao pipeline de CI/CD desde as fases iniciais do projeto reduziria o esforço concentrado no final do desenvolvimento, prática que o grupo pretende adotar em projetos futuros.
- O trabalho em equipe com divisão clara de responsabilidades — unitários, API e E2E para membros diferentes. Permitiu paralelismo na execução e troca de conhecimento sobre diferentes ferramentas e abordagens de teste.

6. Repositório no GitHub

Todo o código-fonte, configurações de CI/CD, coleções Postman e scripts Playwright estão disponíveis no repositório público abaixo. Cada integrante realizou ao menos um commit ou branch conforme exigido nas instruções de entrega (Trab_Final_8 — Grupo 09, apresentação em 01/06).

<https://github.com/Takesh0s/RollCore>

REFERÊNCIAS

- JUNIT TEAM. **JUnit 5 User Guide**. Version 5.10. [S.l.]: JUnit, 2024. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>. Acesso em: 31 maio 2026.
- MICROSOFT. **Playwright Documentation**. [S.l.]: Microsoft, 2024. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 31 maio 2026.
- POSTMAN INC. **Postman Learning Center**. [S.l.]: Postman, 2024. Disponível em: <https://learning.postman.com/>. Acesso em: 31 maio 2026.
- PIVOTAL SOFTWARE. **Spring Boot Reference Documentation**. Version 3.x. [S.l.]: VMware, 2024. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Acesso em: 31 maio 2026.
- MOCKITO PROJECT. **Mockito Framework Site**. [S.l.]: Mockito, 2024. Disponível em: <https://site.mockito.org/>. Acesso em: 31 maio 2026.
- META OPEN SOURCE. **React Documentation**. [S.l.]: Meta, 2024. Disponível em: <https://react.dev/>. Acesso em: 31 maio 2026.
- MICROSOFT. **TypeScript Documentation**. [S.l.]: Microsoft, 2024. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 31 maio 2026.

WIZARDS OF THE COAST. **Player's Handbook: Dungeons & Dragons, 5th Edition.** Renton: Wizards of the Coast, 2014.